

UNIVERSIDAD DE ANTIOQUIA
TEORÍA DE LENGUAJES Y LABORATORIO

Solución Actividad Práctica: Validación de Formulario de Registro con Expresiones Regulares en C++

Docente: Luz Andrea Páez Martínez

Asignatura: Teoría de Lenguajes

Estudiante: Juan Esteban
Mosquera Perea

1. Introducción y Propósito del Taller

El objetivo principal de esta actividad práctica es desarrollar una simulación interactiva en C++ para la validación de campos de un formulario de registro (tomando como referencia el formulario de eventos/reuniones de UdeArroba / Zoom). A través del uso de la librería estándar <regex>, se comprueba de manera rigurosa que cada entrada provista por el usuario cumpla con el formato sintáctico requerido antes de ser aceptada por el sistema.

Lógica de Control Implementada: El programa solicita datos campo por campo mediante funciones reutilizables. Si la entrada satisface la expresión regular definida, el sistema confirma su validez con el mensaje 'Valido' y continúa al siguiente campo. En caso contrario, despliega un mensaje descriptivo de error y repite la solicitud de forma recursiva/cíclica hasta obtener un dato sintácticamente correcto.

2. Análisis de Campos, Reglas de Formato y Expresiones Regulares

A continuación se detallan los 8 patrones de Expresiones Regulares (Regex) implementados en el código de C++, indicando el propósito del campo, la regla sintáctica definida, la sintaxis Regex exacta usada en C++ y ejemplos de casos válidos e inválidos.

Campo / Variable	Expresión Regular (C++)	Regla Sintáctica	Casos Válidos / Inválidos
Nombre (patronNombre)	<code>^[a-zA-Z]{3,}\$</code>	Solo letras (mayúsculas o minúsculas). Longitud mínima de 3 caracteres.	✓ Juan, Yoiner ✗ Jo, J3sus, Ana-
Apellido (patronApellido)	<code>^[a-zA-Z]{2,}\$</code>	Solo letras (mayúsculas o minúsculas). Longitud mínima de 2 caracteres.	✓ Gil, Perez ✗ G, P3rez, Martinez!
Correo (patronCorreo)	<code>^[a-zA-Z0-9]+@[a-zA-Z0-9]+\.[a-zA-Z0-9]+\$</code>	Formato usuario@dominio.extensión.	✓ usuario@udea.edu ✗ usr@dom,

		Solo caracteres alfanuméricos en usuario y dominio.	usr.udea.com
Documento (patronDocumento)	<code>^[0-9]+\$</code>	Cadena compuesta exclusivamente por uno o más dígitos numéricos (0-9).	✓ 1017123456, 123 ✗ 1017A, 10-23, ABC
Dependencia (patronDependencia)	<code>^[a-zA-Z]+\$</code>	Solo letras (mayúsculas/minúsculas) y espacios en blanco.	✓ Artes, Ingeniería ✗ Fac_Ing, UdeA123
Pregrado (patronPregrado)	<code>^[a-zA-Z]+\$</code>	Solo letras y espacios en blanco.	✓ Sistemas, Educación ✗ Ing#Sistemas, P-1
Campus (patronCampus)	<code>^[a-zA-Z]+\$</code>	Solo letras y espacios en blanco.	✓ Posgrados, Uraba ✗ Campus-1, Robledo2
Campo Otro (patronCampoOtro)	<code>^[a-zA-Z0-9]+\$</code>	Permite caracteres alfanuméricos (letras y números) y espacios.	✓ Bioingeniería 2026 ✗ Depto_Artes, A&B

3. Matriz de Pruebas (Casos Válidos e Inválidos)

Para comprobar la efectividad de las expresiones regulares y la robustez del menú interactivo, se evalúan múltiples escenarios de prueba tanto exitosos (válidos) como fallidos (inválidos).

Campo Evaluado	Entrada de Prueba	Resultado Esperado	Justificación Sintáctica
Nombre	Yo	INVÁLIDO	Tiene 2 caracteres. La regla requiere mínimo 3 letras ('^[a-zA-Z]{3,}\$').
Nombre	Yoiner1	INVÁLIDO	Contiene el número '1'. La regla solo permite letras.
Nombre	Yoiner	VÁLIDO	Cumple con 6 letras alfabéticas (> 2).
Apellido	G	INVÁLIDO	Tiene 1 carácter. Requiere mínimo 2 letras ('^[a-zA-Z]{2,}\$').
Apellido	Gomez	VÁLIDO	Cumple con 5 letras alfabéticas.
Correo	yoiner.udea	INVÁLIDO	Falta el símbolo '@' y la estructura de dominio válida.

Correo	yoiner@udea	INVÁLIDO	Falta la extensión del dominio (ej. .com, .edu).
Correo	yoiner@udea.com	VÁLIDO	Cumple estrictamente con 'usuario@dominio.extension'.
Documento	1017ABC	INVÁLIDO	Contiene caracteres no numéricos ('ABC').
Documento	1017123456	VÁLIDO	Cadena compuesta exclusivamente por dígitos ('^[0-9]+\$').
Menú Opción	abc / 3	INVÁLIDO	Se limpia el buffer cin y se rechaza por no ser opción 1 o 2.
Menú Opción	1	VÁLIDO	Entrada válida que inicia el proceso de registro.
Selección Lista	0 / 5	INVÁLIDO	Índice fuera del rango de la lista desplegable.
Selección Lista	2	VÁLIDO	Selecciona correctamente la segunda opción de la lista.

4. Código Fuente Completo en C++

A continuación se adjunta el código fuente completo implementado en C++. El código hace uso de la librería ``<regex>`` para las comprobaciones de formato, así como del manejo seguro de flujos de entrada con ``cin.clear()`` y ``cin.ignore()`` para prevenir bucles infinitos en el menú.

```
#include <iostream>
#include <limits>
#include <regex>
#include <string>
#include <vector>

using namespace std;

// Función para solicitar y validar entradas mediante expresiones regulares
string pedirDato(
    const string& mensaje,
    const regex& patron,
    const string& mensajeError
) {
    string dato;

    while (true) {
        cout << mensaje;
        cin >> dato;

        if (regex_match(dato, patron)) {
            cout << "Valido" << endl;
            return dato;
        }
    }
}
```

```

    }

    cout << mensajeError << endl;
}

// Definición de Patrones con Expresiones Regulares
regex patronNombre("[a-zA-Z]{3,}$");
regex patronApellido("[a-zA-Z]{2,}$");
regex patronCorreo("[a-zA-Z0-9]+@[a-zA-Z0-9]+\\.[a-zA-Z0-9]+$");
regex patronDocumento("[0-9]+$");
regex patronDependencia("[a-zA-Z ]+$");
regex patronPregrado("[a-zA-Z ]+$");
regex patronCampoOtro("[a-zA-Z0-9 ]+$");
regex patronCampus("[a-zA-Z ]+$");

// Función para validar la selección de un menú/lista numerada
string pedirLista(const string& titulo, const vector<string>& opciones) {
    while (true) {
        cout << titulo << endl;
        for (size_t i = 0; i < opciones.size(); ++i) {
            cout << (i + 1) << ". " << opciones[i] << endl;
        }
        cout << "Opcion: ";

        string entrada;
        cin >> entrada;

        if (regex_match(entrada, regex("[1-9][0-9]*$"))) {
            int indice = stoi(entrada) - 1;

            if (indice >= 0 && indice < static_cast<int>(opciones.size())) {
                cout << "Valido" << endl;
                return opciones[indice];
            }
        }

        cout << "Opcion invalida. Debe seleccionar un numero dentro de la lista." << endl;
    }
}

int main() {

    cout << "=====\n";
    cout << "    REGISTRO CURSO DE GESTION\n";
    cout << "=====\n\n";

    while (true) {

        int opcion;

        cout << "\nMENU DE OPCIONES\n";
        cout << "1. Registrarme al curso\n";
        cout << "2. Salir\n";
        cout << "Seleccione una opcion: ";

        while (!(cin >> opcion)) {
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            cout << "\nOpcion invalida. Ingrese solo un numero (1 o 2).\n";
            cout << "Seleccione una opcion: ";
        }
    }
}

```

```

switch (opcion) {

case 1: {

    string nombre = pedirDato(
        "Nombre: ",
        patronNombre,
        "Nombre invalido. Debe comenzar con una letra y tener al menos 3 caracteres
alfanumericos."
    );

    string apellido = pedirDato(
        "Apellido: ",
        patronApellido,
        "Apellido invalido. Debe comenzar con una letra y tener al menos 2 caracteres
alfanumericos."
    );

    string correo = pedirDato(
        "Correo: ",
        patronCorreo,
        "Correo invalido. Debe tener el formato usuario@correo.com"
    );

    vector<string> opcionesTipoDocumento = {"CC", "TI", "PA"};
    string tipoDocumento = pedirLista(
        "Seleccione el tipo de documento:",
        opcionesTipoDocumento
    );

    string documento = pedirDato(
        "Documento: ",
        patronDocumento,
        "Documento invalido. Debe ser un numero."
    );

    vector<string> opcionesVinculacion = {"Docente vinculado", "Externo",
"Administrativo"};
    string vinculacion = pedirLista(
        "Seleccione la vinculacion:",
        opcionesVinculacion
    );

    vector<string> opcionesDependencia = {"Facultad de Artes", "Facultad de Educación",
"Facultad de Ingenieria", "Otro"};
    string dependencia = pedirLista(
        "Seleccione la dependencia:",
        opcionesDependencia
    );

    if (dependencia == "Otro") {
        dependencia = pedirDato(
            "Si no encuentra su dependencia en el listado, puede indicarnos aquí: ",
            patronCampoOtro,
            "Campo invalido. Debe contener solo caracteres alfanumericos y espacios."
        );
    }

    vector<string> opcionesPregrado = {"Ingenieria de Sistemas", "Arte dramatico",
"Licenciatura en Educacion", "Otro"};
    string pregrado = pedirLista(
        "Seleccione el pregrado:",
        opcionesPregrado
    );
}
}

```

```

    );

    string otroPregrado = "Ninguno";
    if (pregrado == "Otro") {
        otroPregrado = pedirDato(
            "Si no encuentra su pregrado en el listado, puede indicarnos aquí: ",
            patronCampoOtro,
            "Campo invalido. Debe contener solo caracteres alfanumericos y espacios."
        );
    }

    string otroPosgrado = "Ninguno";
    if (pregrado == "Otro") {
        otroPosgrado = pedirDato(
            "Si no encuentra su posgrado en el listado, puede indicarnos aquí: ",
            patronCampoOtro,
            "Campo invalido. Debe contener solo caracteres alfanumericos y espacios."
        );
    }

    vector<string> opcionesCampus = {"Ciudad Universitaria (Medellín)", "Uraba",
"Posgrados"};
    string campus = pedirLista(
        "Seleccione el campus:",
        opcionesCampus
    );

    cout << "\n=====\\n";
    cout << "          Datos registrados\\n";
    cout << "=====\\n\\n";

    cout << "Nombre: " << nombre << endl;
    cout << "Apellido: " << apellido << endl;
    cout << "Correo: " << correo << endl;
    cout << "Tipo de documento: " << tipoDocumento << endl;
    cout << "Documento: " << documento << endl;
    cout << "Vinculacion: " << vinculacion << endl;
    cout << "Pregrado: " << pregrado << endl;
    cout << "Otro pregrado: " << otroPregrado << endl;
    cout << "Otro posgrado: " << otroPosgrado << endl;
    cout << "Dependencia: " << dependencia << endl;
    cout << "Campus: " << campus << endl;
    cout << "\\nRegistro exitoso." << endl;

    break;
}

case 2:
    cout << "\\nSaliendo del programa..." << endl;
    return 0;

default:
    cout << "\\nOpcion invalida. Seleccione 1 o 2." << endl;
    break;
}
}

return 0;
}

```

5. Ejemplo de Simulación de Ejecución en Consola

A continuación se ilustra la salida del programa durante un flujo interactivo en el que se prueban reiterados intentos inválidos hasta ingresar los datos correctos:

```
=====
      REGISTRO CURSO DE GESTION
=====

MENU DE OPCIONES
1. Registrarme al curso
2. Salir
Seleccione una opcion: 1
Nombre: Yo
Nombre invalido. Debe comenzar con una letra y tener al menos 3 caracteres alfanumericos.
Nombre: Yoiner
Valido
Apellido: G
Apellido invalido. Debe comenzar con una letra y tener al menos 2 caracteres alfanumericos.
Apellido: Perez
Valido
Correo: yoiner.correo
Correo invalido. Debe tener el formato usuario@correo.com
Correo: yoiner@udea.edu
Valido
Seleccione el tipo de documento:
1. CC
2. TI
3. PA
Opcion: 1
Valido
Documento: 1017abc
Documento invalido. Debe ser un numero.
Documento: 1017123456
Valido
Seleccione la vinculacion:
1. Docente vinculado
2. Externo
3. Administrativo
Opcion: 2
Valido
Seleccione la dependencia:
1. Facultad de Artes
2. Facultad de Educación
3. Facultad de Ingenieria
4. Otro
Opcion: 3
Valido
Seleccione el pregrado:
1. Ingenieria de Sistemas
2. Arte dramatico
3. Licenciatura en Educacion
4. Otro
Opcion: 1
Valido
Seleccione el campus:
1. Ciudad Universitaria (Medellín)
2. Uraba
3. Posgrados
Opcion: 1
Valido
```

=====

Datos registrados

=====

Nombre: Yoiner
Apellido: Perez
Correo: yoiner@udea.edu
Tipo de documento: CC
Documento: 1017123456
Vinculacion: Externo
Pregrado: Ingenieria de Sistemas
Otro pregrado: Ninguno
Otro posgrado: Ninguno
Dependencia: Facultad de Ingenieria
Campus: Ciudad Universitaria (Medellín)

Registro exitoso.